03 - Loops

The programs we have written so far run each line once. This notebook is about programs that **repeat**:

- `for` loops and `range`
- The accumulator pattern
- Searching with `break`, skipping with `continue`
- `enumerate` and `zip`
- Looping over dictionaries
- `while` loops
- Nested loops
- Splitting and joining strings
- List comprehension

Roughly the first half is about repeating something once per item, and the second half is about repeating something until you are done.

Why we need loops

In the first set of home exercises you printed a receipt for three items. The data was in three lists, and the item at each position belonged together — `items[0]` was sold in quantity `quantities[0]` at price `unit_prices[0]`.

With no way to repeat anything, the only option was to write the same line out three times.

In [ ]:
```
items = ["Espresso machine", "Coffee beans", "Oat milk"]
quantities = [1, 2, 3]
unit_prices = [4999.00, 149.90, 24.50]

print(f"{items[0]:<20}{quantities[0]:>3} x {unit_prices[0]:>9.2f}")
print(f"{items[1]:<20}{quantities[1]:>3} x {unit_prices[1]:>9.2f}")
print(f"{items[2]:<20}{quantities[2]:>3} x {unit_prices[2]:>9.2f}")
```

But what if you had 300 items? Then you would have to write 300 lines!

Here is the same receipt written with a loop. Read it, but do not worry about the syntax yet.

In [ ]:
```
for i in range(len(items)):
    print(f"{items[i]:<20}{quantities[i]:>3} x {unit_prices[i]:>9.2f}")
```

Two lines, and it no longer cares how many items there are.

Loading [MathJax]/extensions/Safe.js

This is the third and last of the control structures. We have used **sequential** control since the beginning — statements running one after another — and **selection** to pick one block of code over another. This notebook is on **iterative** control: running the same block of code again and again.

The `for` loop

A `for` loop runs a block of code once for each item in a sequence.

```
for <loop variable> in <sequence>:
    <statements>
```

The header starts with `for`, names a **loop variable**, gives a sequence, and ends with a **colon**. The block underneath is **indented**, exactly as it was for `if` statements. On each pass the loop variable is attached to the next item in the sequence, and the block runs again.

A string is a sequence of characters, so we can loop over one straight away.

In [ ]:
```
for letter in "Python":
    print(letter)
```

A list is a sequence too.

In [ ]:
```
names = ["Ole", "Jenny", "Chang", "Jonas"]

for name in names:
    print(name)
```

> 💡 **Tip:** Name the loop variable after **one item** of the sequence — `name` for a list of names, `city` for a list of cities. A single letter such as `i` is conventional only when the variable holds a *position* rather than an item.

Any statements at all can go in the block, not only `print`.

In [ ]:
```
for name in names:
    print(f"{name.upper()} has {len(name)} letters")
```

The loop variable does not disappear when the loop ends. It keeps whatever it was given on the last pass, which is occasionally useful and quite often confusing.

In [ ]:
```
for name in names:
    print(name)

ter the loop, name is still {name}")
```

Loading [MathJax]/extensions/Safe.js

The loop variable is a name, not a slot in the list

This is the single most common loop bug. Remember that assignment attaches a **name** to an object, it does not write into a container.

The loop below looks like it raises every price by 25%.

In [ ]:
```
prices = [199, 249, 149]

for price in prices:
    price = price * 1.25

print(prices)
```

Nothing changed. On each pass `price` is attached to the next item in the list; assigning to `price` detaches that name and points it at a new number. The list was never involved.

To actually change things you either build a new list or write back into the list by position (more on this later).

`range`

Very often we want to repeat something a fixed number of times rather than walk over a list Python already has. The `range` function builds the sequence of whole numbers for us.

In [ ]:
```
range(5)
```

Notice that `range` does not build anything — it produces the numbers one at a time, as the loop asks for them.

If we want to *see* the numbers, we can ask for the list explicitly.

In [ ]:
```
list(range(5))
```

It starts at **0**, and it stops **before** 5 — the same rule as slicing. So `range(5)` produces five numbers, and never includes the number you wrote.

In [ ]:
```
for i in range(5):
    print(i)
```

Given two arguments, `range` starts somewhere other than 0.

In [ ]:
```
for i in range(1, 6):
    print(i)
```

Loading [MathJax]/extensions/Safe.js

And given three, the last one is the **step**: how far to move each time. A negative step counts downwards.

In [ ]:
```
print(list(range(0, 10, 2)))
print(list(range(10, 0, -1)))
```

> 💡 **Tip:** The stop value is the off-by-one trap. To count from 1 to 10 you must write `range(1, 11)`. Whenever a loop does one pass too few, look at the stop value first.

Your turn

Use a `for` loop and `range` to display the seven times table, one row per line:

```
7 x 1 = 7
7 x 2 = 14
...
7 x 10 = 70
```

In [ ]:
```

```

The accumulator pattern

Displaying things as we go is fine, but usually we want to *keep* a result.

In [ ]:
```
for num in range(1, 6):
    square = num ** 2

print(square)
```

Only the last value survived. `square` is assigned a new value on every pass, so each result overwrites the one before it and all we are left with is the last.

The fix is the **accumulator pattern**, and it has three parts:

1. **initialize** a variable *before* the loop, to a neutral starting value
2. **update** it inside the loop
3. **use** it after the loop has finished

In [ ]:
```
total = 0

for num in range(1, 6):
    total = total + num

print(total)
```

Loading [MathJax]/extensions/Safe.js

`total = total + num` appears so often that Python has a shorthand for it: `+=`, called an **augmented assignment**. The two lines below mean exactly the same thing.

In [ ]:
```
total = 0

for num in range(1, 6):
    total += num

print(total)
```

`-=`, `*=` and `/=` exist as well and work the same way.

> ⚠️ **Warning:** The initialization goes **before** the `for`, never inside it. `total = 0` inside the loop resets the total on every pass, and the answer you get is whatever the last item contributed.

An accumulator does not have to be a sum. We can combine it with an `if` statement to count the number of occurrences.

In [ ]:
```
temperatures = [4.2, -3.1, 0.0, -8.5, 12.3, -0.5, 7.8]

freezing_days = 0

for temperature in temperatures:
    if temperature < 0:
        freezing_days += 1

print(f"{freezing_days} days below zero")
```

And starting from an empty list instead of a zero, we can store operations in a list with `append()`. This is how we build a new list out of an old one.

In [ ]:
```
prices = [199, 249, 149]
new_prices = []

# Append prices to new list
for price in prices:
    new_prices.append(price * 1.25)

print(prices)
print(new_prices)
```

Alternatively, we can use the `range` function to write back into a list by position. In that case, we modify the original list instead of storing outputs in a new list.

In [ ]:
```
prices = [199, 249, 149]

# Update prices in original list
ange(len(prices)):
```

Loading [MathJax]/extensions/Safe.js

```
    prices[i] = prices[i] * 1.25

print(prices)
```

Your turn

A small business has recorded these transactions, where a negative amount is a refund:

```
transactions = [1250.00, -340.50, 899.00, 120.00, -75.25, 2100.00]
```

Using a single loop, work out the total value of the transactions and how many of them were refunds. Display both.

In [ ]:

Stopping early and skipping ahead

By default a `for` loop visits every item. Two keywords change that: `break` and `continue`

`break` leaves the loop immediately. Nothing after it in the block runs, and no further items are visited. That is exactly what we want when we are **searching**: once you have found the thing, carrying on is a waste.

In [ ]:
```
revenues = [120, 340, 55, 890, 210, 950, 670]

for revenue in revenues:
    if revenue > 900:
        print(f"First revenue above 900: {revenue}")
        break
```

There is a hole in that program, though. If no revenue is above 900, the `print` never runs and the program says nothing at all — which looks identical to it not having run.

The standard repair is a **flag**: a boolean variable recording whether the search succeeded.

In [ ]:
```
codes = ["SE-114", "DK-201", "NO-450", "FI-330"]

found = False

for code in codes:
    if code.startswith("XX"):
        print(f"Found: {code}")
        found = True
        break
```

Loading [MathJax]/extensions/Safe.js

```
if not found:
    print("No matching code.")
```

`continue` is the milder one of the two. It abandons the current pass and jumps straight to the next item, without leaving the loop.

In [ ]:
```
values = [10, -3, 0, 7, -1, 4]

for value in values:
    if value <= 0:
        continue

    print(value)
```

That loop does the same thing as putting the `print` inside `if value > 0:`. `continue` earns its place when the rest of the block is long: it deals with the awkward items up front and leaves the main work at one level of indentation instead of two.

Your turn

A list of booking references has arrived:

```
references = ["DK-1180", "SE-2240", "NO-7830", "NO-1120"]
```

Write a loop that displays the **first** reference beginning with `"NO"`, and stops looking once it has found one. If the list contains no such reference, it should display a message saying so instead — check that by removing the two `NO` references and running it again.

In [ ]:
```

```

Knowing where you are

Looping over items is the normal case. Sometimes you need the **position** as well, and sometimes you need to walk over two lists in step. There are three ways to do it, and only the first one is clumsy.

In [ ]:
```
cities = ["Oslo", "Bergen", "Trondheim"]

for i in range(len(cities)):
    print(f"{i + 1}. {cities[i]}")
```

That works — `range(len(cities))` produces the positions, and we index into the list ourselves. You will see it in other people's code, so it is worth being able to read.

Loading [MathJax]/extensions/Safe.js

`enumerate` does the same job without the bookkeeping. It hands back the position **and** the item on each pass, so we give the loop two variable names separated by a comma.

In [ ]:
```
for i, city in enumerate(cities):
    print(f"{i + 1}. {city}")
```

Since numbering from 1 is so common, `enumerate` can start its count wherever you like.

In [ ]:
```
for number, city in enumerate(cities, start=1):
    print(f"{number}. {city}")
```

`zip` solves the other problem: several lists whose items belong together by position. It walks them in step and hands back one item from each.

In [ ]:
```
quantities = [3, 5, 2]

for city, quantity in zip(cities, quantities):
    print(f"{quantity} orders from {city}")
```

> 💡 **Tip:** Reach for plain `for item in items` first. Use `enumerate` when you genuinely need the position, and `zip` when multiple lists belong together. Note that `zip` stops as soon as the **shortest** list runs out, so it will quietly ignore the tail of a longer one.

Your turn

The receipt data from the start of this notebook:

```
items = ["Espresso machine", "Coffee beans", "Oat milk"]
quantities = [1, 2, 3]
unit_prices = [4999.00, 149.90, 24.50]
```

Use `zip` to display one line per item, showing the name, the quantity and the unit price. Do not index into the lists.

In [ ]:
```

```

Looping over dictionaries

A dictionary can be looped over as well, but it has both keys and values, so we have to say which we want. Looping over the dictionary itself gives the **keys**.

Loading [MathJax]/extensions/Safe.js

In [ ]:
```
capitals = {"Norway": "Oslo", "Sweden": "Stockholm", "Denmark": "Copenhagen"

for country in capitals:
    print(country)
```

Given a key we can look up its value, exactly as before.

In [ ]:
```
for country in capitals:
    print(f"The capital of {country} is {capitals[country]}.")
```

If the keys are of no interest, `.values()` skips straight to the values.

In [ ]:
```
for capital in capitals.values():
    print(capital)
```

And `.items()` gives both at once. It hands over each key and value as a **pair**.

In [ ]:
```
for pair in capitals.items():
    print(pair)
```

Those are tuples — the immutable version of the list. We rarely want them in that form, so we write **two** names in the loop header and Python splits each pair between them. This is the same thing that happened with `enumerate` and `zip`.

In [ ]:
```
for country, capital in capitals.items():
    print(f"The capital of {country} is {capital}.")
```

`while` loops

A `for` loop repeats once per item. A `while` loop repeats for as long as a **condition** is true, and it has no idea how many passes that will take.

```
while <condition>:
    <statements>
```

The condition is an ordinary condition of the kind we built last time — comparisons, `in`, `and`, `or`, truthiness, all of it applies unchanged.

A `while` loop has the same three parts as the accumulator pattern:

- something is **initialized** before the loop,
- **tested** in the header,
- and **updated** inside the block.

In [ ]:
```
count = 1          # initialize

t <= 5:  # test
```

Loading [MathJax]/extensions/Safe.js

```
    print(count)
    count += 1     # update
```

The condition is tested *before* each pass, including the first. If it is false to begin with, the block never runs at all — which is not an error, just an empty loop.

In [ ]:
```
count = 10

while count <= 5:
    print(count)
    count += 1

print("Done.")
```

Infinite loops

If nothing inside the block can ever make the condition false, the loop never ends. Here are the three ways to write one — do **not** run these, they are printed as text on purpose.

```
# 1. Nothing updates the variable in the condition
count = 1
while count <= 5:
    print(count)

# 2. The wrong variable is updated
count = 1
total = 0
while count <= 5:
    total += count

# 3. The condition can never become false
count = 1
while count > 0:
    count += 1
```

The first is by far the most common, and it is the reason the update line matters as much as the condition.

> ⚠️ **Warning:** Sooner or later you will run one by accident. The cell will simply never finish, and Positron will keep showing it as running. Stop it with the **Stop cell execution** button beside the cell. If the notebook stays confused afterwards, restart the kernel and run the cells above it again.

When the number of passes is genuinely unknown

This is what `while` is *for*. A machine worth 250 000 kroner loses 20% of its value each year. How many years until it is worth less than 100 000?

Loading [MathJax]/extensions/Safe.js

You cannot answer that before running it — it depends on the starting value, the rate and the threshold — so there is no sequence for a `for` loop to walk over.

In [ ]:
```
value = 250_000
years = 0

while value >= 100_000:
    value = value * 0.8
    years += 1

print(f"After {years} years the value is {value:,.0f} kroner.")
```

Repeating until the user gets it right

Every program you wrote in the last set of home exercises gave up when the input was wrong: it displayed a message and stopped. That is a poor experience, and now we can do better — ask again, and keep asking.

The cell below asks you to type something. The prompt appears at the **top of the window**, and the cell keeps running until you answer it. Enter something invalid first to watch the loop go round, then enter `C` or `F` to get out.

In [ ]:
```
choice = input("Enter C to convert to Celsius, or F to convert to Fahrenheit

while choice not in ("C", "F"):
    print("That is not a valid choice.")
    choice = input("Enter C to convert to Celsius, or F to convert to Fahrer

print(f"You selected: {choice}")
```

Notice the duplication: the prompt is written twice, once to get the first answer and once to get every answer after it. That is the price of testing the condition at the top.

The alternative is a loop that never ends on its own, with a `break` in the middle.

```
while True:
    choice = input("Enter C to convert to Celsius, or F to convert
to Fahrenheit: ").strip().upper()

    if choice in ("C", "F"):
        break

    print("That is not a valid choice.")
```

Your turn

A savings account holds 25 000 kroner and grows by 3% each year.

Loading [MathJax]/extensions/Safe.js

> Use a `while` loop to work out how many whole years it takes for the balance to pass 40 000 kroner. Display the number of years and the final balance.

In [ ]:

`for` or `while`?

Most tasks have a natural answer.

Use a `for` loop when	Use a `while` loop when
you are walking over a sequence you already have	there is no sequence to walk over
you know the number of passes in advance	the number of passes depends on what happens inside the loop
`range` can produce the numbers you need	the loop waits on something outside the program, such as a person typing

Where both would work, prefer `for`. A `while` loop that counts to the length of a list is a `for` loop with three extra lines of bookkeeping and one chance to forget the update.

In [ ]:
```
cities = ["Oslo", "Bergen", "Trondheim"]

i = 0
while i < len(cities):
    print(cities[i])
    i += 1
```

`for city in cities:` says the same thing in one line, cannot run off the end of the list, and cannot loop forever.

Nested loops

A loop can go inside another loop, just as an `if` could go inside another `if`.

```
for <outer variable> in <sequence>:
    <outer block>
    for <inner variable> in <sequence>:
        <inner block>
```

The important thing to hold on to: the **inner loop runs all the way through on every single pass of the outer loop**. The two loop variables must have different names, or the inner one will overwrite the outer one.

Loading [MathJax]/extensions/Safe.js

In [ ]:
```
for day in ["Mon", "Tue"]:
    for slot in ["morning", "afternoon"]:
        print(f"{day} {slot}")
```

The outer block can hold more than just the inner loop, and **where** a statement sits decides how often it runs.

The code below is the same nested loop with one extra `print` added to the outer block, above the inner loop — so it runs once per day rather than once per slot

In [ ]:
```
for day in ["Mon", "Tue", "Wed"]:
    print(f"{day}:")

    for slot in ["morning", "afternoon"]:
        print(f"    {slot}")
```

Three days, so three headings and six indented lines. Move that print down into the inner block and you would get the heading six times instead

Looping over a nested list

Nesting is how we get at data that is itself nested. Each item of the outer list is a list in its own right, so the outer loop hands the inner loop something to walk over.

In [ ]:
```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

for row in matrix:
    for value in row:
        print(value, end=" ") # use end=" " so rows stay on one line

    print()                   # add line break at the end of each row
```

A dictionary whose values are lists nests the same way, and it is a natural place to put an accumulator inside the inner loop.

In [ ]:
```
grades = {
    "Alice": [85, 90, 78],
    "Bob": [92, 88, 95],
    "Charlie": [70, 75, 80]
}

for student, scores in grades.items():
    total = 0 # initialize total for each student

    for score in scores:
        total += score
```

Loading [MathJax]/extensions/Safe.js

```
    print(f"{student}: {total / len(scores):.1f}")
```

> 💡 **Tip:** A nested loop is right when you genuinely need every combination, or when the data is itself nested. It is wrong when the inner loop is doing something Python already has a name for. In the example above: the inner loop over scores adds up a list, and `sum(scores)` gives that with no inner loop at all.

`break` only leaves the loop it is in

`break` leaves the loop it is written in and nothing more.

In [ ]:
```
matrix = [
    [1, 2, 3],
    [4, 99, 6],
    [7, 8, 9]
]

for row in matrix:
    for value in row:
        if value == 99:
            print("found 99, stopping")
            break

        print(value, end=" ")

    print()
```

To leave both loops, set a flag in the inner loop and test it in the outer one.

In [ ]:
```
found = False

for row in matrix:
    for value in row:
        if value == 99:
            found = True
            break

    if found:
        break

print(f"Stopped early: {found}")
```

Your turn

Each sublist holds one student's scores on three tests:

Loading [MathJax]/extensions/Safe.js

```
table = [
    [85, 91, 89],
    [78, 81, 86],
    [62, 75, 77]
]
```

Use a nested loop to display one line per student, showing every score, with a `*` after any score above 80:

```
Student 1: 85* 91* 89*
Student 2: 78 81* 86*
Student 3: 62 75 77
```

Keeping a row on one line needs `end=" "`, and the newline at the end of each row needs a `print()` of its own.

In [ ]:

Splitting and joining strings

Text rarely arrives one value at a time. It arrives as a line, with the values inside it, and the first job is to break it apart. That is `.split()`.

In [ ]:
```
line = "Oslo,Bergen,Trondheim"

parts = line.split(",")

print(parts)
print(type(parts))
```

`.split()` returns a **list** that we can now easily loop over to access each word.

In [ ]:
```
for city in line.split(","):
    print(city)
```

With an argument, and without

Called with no argument at all, `.split()` behaves differently — it splits on any run of whitespace (spaces, tabs, newlines) and throws the empties away.

In [ ]:
```
sentence = "  the   quick brown   fox  "

print(sentence.split(" "))  # split on a single whitespace
print(sentence.split())     # default (split on *any* whitespace)
```

The separator itself is never included in the pieces, and nothing else is cleaned up either.

Loading [MathJax]/extensions/Safe.js

In [ ]:
```
raw = "norway , GERMANY, brazil ,france"

print(raw.split(","))
```

Every piece has kept its stray spaces and its odd capitalization. Splitting is only the first half of the job; a loop does the rest.

In [ ]:
```
cleaned = []

for part in raw.split(","):
    cleaned.append(part.strip().capitalize())

print(cleaned)
```

One edge case is worth seeing once, because it is not what people expect.

In [ ]:
```
print("".split(","))
```

Splitting an empty string gives a list containing **one** empty string, not an empty list.

`.join()`

`.join()` is `.split()` backwards: it takes a list of strings and glues them into one string. The string you call it *on* is the glue.

In [ ]:
```
cities = ["Oslo", "Bergen", "Trondheim"]

print(", ".join(cities))
print(" | ".join(cities))
print("".join(["4", "7", "1", "9"]))
```

The last one is the useful trick for turning a list of characters or digits into a single string.

Notice that `.join()` only joins **strings**, though. A list of numbers has to be converted first.

In [ ]:
```
# TypeError
digits = [4, 7, 1, 9]

print("".join(digits))
```

In [ ]:
```
digits = [4, 7, 1, 9]

text = ""

for digit in digits:
    text += str(digit)

print(text)
```

Loading [MathJax]/extensions/Safe.js

Your turn

A messy list of city names has arrived as a single string:

```
raw = "  bergen , OSLO,trondheim , Stavanger  "
```

Display the names on one line, separated by `  –  `, with no stray spaces and each name capitalized — like this:

```
Bergen – Oslo – Trondheim – Stavanger
```

In [ ]:

List comprehension

Building a new list out of an old one is so common that Python has a shorthand for it. Here is the pattern we already know.

In [ ]:
```
revenues = [120, 340, 55, 890]

doubled = []

for revenue in revenues:
    doubled.append(revenue * 2)

print(doubled)
```

And here it is as a **list comprehension**: the same three parts — the new list, the loop, the expression — folded into one line.

```
[<expression> for <item> in <sequence>]
```

The square brackets are doing real work: they say the result is a list. Read it from the middle outwards — *for each revenue in revenues, put revenue times two in the list*.

In [ ]:
```
doubled = [revenue * 2 for revenue in revenues]

print(doubled)
```

An `if` written **after** the `for` filters. Items that fail the test are left out, so the new list is shorter than the old one.

In [ ]:
```
high = [revenue for revenue in revenues if revenue > 200]

print(high)
```

Loading [MathJax]/extensions/Safe.js

Notice that an `if` / `else` written **before** the `for` does something quite different. It is not a filter — it chooses between two values for every item, so the new list is the same length as the old one.

In [ ]:
```
labels = ["high" if revenue > 200 else "low" for revenue in revenues]

print(labels)
```

Two jobs, two positions, and they are not interchangeable. A filter has no `else`, because there is nothing to put in the list for an item it rejects — and Python says so.

In [ ]:
```
# SyntaxError
labels = [revenue for revenue in revenues if revenue > 200 else 0]
```

Your turn

A list of temperatures in Celsius:

```
temperatures = [4.2, 18.5, 11.0, 22.3, 8.7, 16.1]
```

Write a **single** list comprehension that produces only the temperatures above 10, converted to Fahrenheit with `F = 9 / 5 * C + 32`.

In [ ]:
```

```

When a comprehension is not the answer

A comprehension is shorter than the loop it replaces, and shorter is often better. It is not always better, and it is worth knowing where the line is.

A comprehension exists to produce a list. If you are not keeping the result, you are using the wrong tool.

In [ ]:
```
cities = ["Oslo", "Bergen", "Stavanger"]

result = [print(city) for city in cities]

print(result)
```

A comprehension cannot carry anything from one pass to the next. A running total is the standard example: each value depends on all the values before it, and a comprehension has nowhere to keep it.

In [ ]:
```
revenues = [120, 340, 55, 890]

[]
total = 0
```

Loading [MathJax]/extensions/Safe.js

```
for revenue in revenues:
    total += revenue
    running.append(total)

print(running)
```

And a comprehension can nest, but not always readably.

In [ ]:
```
matrix = [[1, 2, 3], [4, 5, 6]]

flat = [value for row in matrix for value in row]

print(flat)
```

That is legal, and the two `for` clauses appear in the same order as the nested loop they replace. It is also the point where a comprehension stops being clearer than the four lines it saved.

> 💡 **Tip:** Use a comprehension when it builds one list, fits comfortably on one line, and can be read aloud in a single breath. Otherwise write the loop. Clarity — not brevity — is the goal.

Additional resources

- The Python Tutorial — section 4 covers `for`, `range`, `break` and `continue`
- More on lists — section 5.1.3 is the official account of list comprehensions
- String methods — the exact wording of `.split()` and `.join()`, including what happens with no argument
- Automate the Boring Stuff with Python — chapter 3 covers `for` and `while` loops, free to read online
- SKL401 — lessons 2.3, 2.4 and 2.5 cover loops and comprehensions more briefly, and 2.9 covers string operations

Next week: functions — packaging a piece of code under a name, so that we can use it again without writing it out again.

Loading [MathJax]/extensions/Safe.js